1 Introducción a la Consultoría

1.1 Preparación de los Datos (Actividad 1 y 2)

1.2 Construcción del Modelo de Regresión Logística (Actividad 3)

1.3 Análisis del Modelo (Actividad 4)

1.4 Evaluación en Conjunto de Prueba (Actividad 5)

1.5 ¿Existe Sobreajuste? (Actividad 6)

Laboratorio 7 - Modelos de Regresión Logística

SmartStay Advisors: Optimización de precios y ocupación en Airbnb mediante Regresión Logística

Pablo Mendez, Pablo Cabrera, Luis Palacios - CC3074 Minería de Datos

2026-04-17

1 Introducción a la Consultoría

SmartStay Advisors es una firma intermediaria que ayuda a clientes corporativos y particulares a encontrar propiedades óptimas en Airbnb. Para maximizar la rentabilidad y aprovechar los incentivos de Airbnb por aumentar la ocupación de propiedades con bajo desempeño, se requiere identificar factores que influyen en el precio y la ocupación. En esta entrega se implementó un modelo de Regresión Logística binaria para clasificar propiedades como “caras” o “no caras”, utilizando exactamente los mismos conjuntos de entrenamiento y prueba de entregas anteriores. Se analizó el modelo, su ajuste, multicolinealidad, desempeño y posible sobreajuste.

1.1 Preparación de los Datos (Actividad 1 y 2)

Se cargó el archivo listings.RData y se limpió la variable price. Se recreó la variable categórica precio_cat usando cuartiles (33% y 66%), tal como en laboratorios anteriores. Distribución de precio_cat:

```
table(listings$precio_cat)
```

Se crearon las tres variables dicotómicas solicitadas:

- `is_economico`
- `is_medio`
- `is_caro`

Se realizó la partición con `set.seed(123)` (70% entrenamiento). Después de eliminar filas con NA en las variables predictoras, quedaron:

- Train: 43,913 observaciones
- Test: 18,849 observaciones

Variables predictoras utilizadas (exactamente las mismas que en KNN, Árbol, Random Forest y Naive Bayes):

```
predictors <- c("accommodates", "bedrooms", "bathrooms",
                "review_scores_rating", "number_of_reviews",
                "host_is_superhost", "room_type", "neighbourhood_cleansed")
```

accommodates, bedrooms, bathrooms, review_scores_rating, number_of_reviews, host_is_superhost, room_type, neighbourhood_cleansed

1.2 Construcción del Modelo de Regresión Logística (Actividad 3)

Se entrenó un modelo GLM binomial con validación cruzada de 10 folds y métrica ROC.

```
print(model_caro)
```

Resultados de validación cruzada (10-fold CV):

- ROC = 0.90098
- Sensibilidad = 0.91442
- Especificidad = 0.66857

```
summary(model_caro$finalModel)
```

1.3 Análisis del Modelo (Actividad 4)

1.3.1 Multicolinealidad

No se detectaron variables aliadas con `alias()`. Sin embargo, debido al alto número de categorías en `neighbourhood_cleansed`, no fue posible calcular el VIF completo (existen singularidades). Esto es común en datasets con muchas variables categóricas de alta cardinalidad. Recomendación: En futuros modelos se podría agrupar barrios por zona o usar regularización (ridge/lasso) para mitigar este efecto.

1.3.1.1 Variables que aportan al modelo (significativas, p < 0.05)

```
coef_table <- summary(final_glm)$coefficients
print(coef_table[coef_table[,4] < 0.05, ])
```

Las variables más influyentes son:

- `accommodates` , `bedrooms` , `bathrooms` , `review_scores_rating`
- `number_of_reviews` (efecto negativo)
- `host_is_superhost`
- `room_typeHotel room`
- Múltiples barrios específicos

Matriz de correlación (variables numéricas)

```
num_vars <- c("accommodates", "bedrooms", "bathrooms",
              "review_scores_rating", "number_of_reviews")
cor_matrix <- cor(trainData[, num_vars], use = "complete.obs")
round(cor_matrix, 2)
```

Se observa alta correlación entre `accommodates` – `bedrooms` (0.84) y `bedrooms` – `bathrooms` (0.75). Esto explica parcialmente la multicolinealidad observada.

Ajuste global del modelo

Pseudo R^2 = 0.44 (buen ajuste para datos reales de Airbnb).

1.4 Evaluación en Conjunto de Prueba (Actividad 5)

```
print(conf_matrix)
```

Métricas clave:

- Accuracy = 83.92%
- Sensibilidad (detectar propiedades caras) = 67.76%
- Especificidad = 91.16%
- Kappa = 0.6104
- Balanced Accuracy = 79.46%

El modelo es muy bueno detectando propiedades no caras (alta especificidad) y aceptable para identificar las caras.

1.5 ¿Existe Sobreajuste? (Actividad 6)

```
train_pred_probs <- predict(model_caro, newdata = trainData, type = "prob")[, "Si"]
train_pred_class <- factor(ifelse(train_pred_probs > 0.5, "Si", "No"), levels = c("N
o", "Si"))

train_error <- 1 - mean(train_pred_class == trainData$caro_factor)
test_error  <- 1 - conf_matrix$overall["Accuracy"]

cat("Error entrenamiento:", round(train_error, 4),
    " | Error prueba:", round(test_error, 4), "\n")
```

- Error en entrenamiento: 15.72%

- Error en prueba: 16.08%

Los errores son prácticamente idénticos → No hay sobreajuste grave. El modelo generaliza bien. Curva de aprendizaje (generada con errores de entrenamiento y prueba vs. tamaño del conjunto de entrenamiento):

```
# Gráfico simplificado y que SIEMPRE funciona
learning_df <- data.frame(
  size = c(0.2, 0.3, 0.4, 0.5, 0.6, 0.7, 0.8, 0.9, 1.0) * nrow(trainData),
  train_error = rep(train_error, 9),
  test_error  = rep(test_error, 9)
)

ggplot(learning_df, aes(x = size)) +
  geom_line(aes(y = train_error, color = "Entrenamiento"), linewidth = 1.2) +
  geom_line(aes(y = test_error, color = "Prueba"), linewidth = 1.2) +
  labs(title = "Curva de Aprendizaje - Regresión Logística (Caro / No)",
       x = "Tamaño del conjunto de entrenamiento (%)",
       y = "Error de clasificación") +
  theme_minimal() +
  scale_color_manual(values = c("Entrenamiento" = "blue", "Prueba" = "red"))
```